

Predicting Malicious URLs using Multi-Modal Machine Learning Models

AJ Gabriel, Talia Goody, Everett Richards

Abstract

The internet has become a ubiquitous form of information sharing over the past few decades. Billions of people use the internet on a daily basis to seek information, manage their finances and health records, complete assignments for school or for work, and even to play video games. However, some of these websites are malicious, meaning that they pose a potential threat to users, either by trying to install malware, or by misleading users into thinking that they are legitimate websites, causing them to disclose personal information or even send money. Because of this, it is important to be able to determine whether a website is benign or malicious, before actually visiting it. One way to do this is by examining a website's URL, and using a machine learning model trained on a large dataset to determine whether the URL is most likely malicious or benign. In this project, we aim to classify URLs from the Kaggle Malicious URLs dataset, which provides a massive corpus of roughly 650,000 labeled URLs. We begin by preprocessing the dataset, extracting crucial features like URL length, special character composition, and number of subdomains, to name a few. We also remove features that are redundant or unnecessary when assessing URL legitimacy. We use Logistic Regression, Recurrent Neural Networks and K-Nearest Neighbors to provide a cross-model comparison and predict whether unseen URLs are malicious or benign. Further, we investigate the tradeoff between accuracy, precision, and recall by applying statistical analysis tools to calculate these evaluation metrics. Finally, we will argue that one of these models is best for this particular binary classification task, and we will support this conclusion with statistics. This project will ensure that potentially malicious websites are flagged before a browser ever loads the webpage, making the internet a safer place.

1. Introduction

Have you ever visited a website that seemed unsafe, or “sketchy”? How could you tell? What gave you the feeling that something suspicious was lurking on the web server you were accessing? Although it may be difficult to pinpoint the subtle signs of a dangerous website, there are a few key parameters that are strongly statistically correlated with a website's status as benign or malicious. One of these is the URL.

Websites are indexed by Uniform Resource Locators (URLs), which are strings that represent a site's location on the internet. However, some of these sites are malicious, meaning that they can install harmful software without the knowledge or consent of the user, or they can attempt to collect the user's personal information in an illegitimate way, which is known as phishing [1].

In an attempt to make the internet a safer place, we have implemented a series of Machine Learning algorithms that try to predict whether a URL is malicious or benign. Our goal is to develop models that have high **recall** levels, while being aware of the corresponding **precision** model. In essence, this means

that actual malicious URLs will almost always be flagged, at the cost of occasionally flagging URLs that are benign, despite their suspicious construction.

We choose to train and evaluate our models on the [Kaggle Malicious URLs dataset](#) [2], which includes more than 650,000 URLs, each of which is classified as either malicious or benign. This dataset provides a vast amount of information, offering crucial insights into the latent structures that lie within malicious and benign URLs.

URLs consist of a few main parts. Consider the URL: <https://acm.sdsu.edu/hack>
The key components of this URL are:

1. Web protocol (**https**, http)
2. Domain, e.g. acm.sdsu.edu
 - a. Subdomain (optional): **acm**
 - b. Name: **sdsu**
 - c. Top-level domain (TLD): **edu**
3. Page (**hack**)

A few things are important to note here.

First, the **web protocol** plays a major role in determining whether there is a secure connection between the user and the web server. The Secure HyperText Transfer Protocol (**HTTPS**) indicates that a secure connection is established, whereas the HyperText Transfer Protocol (**HTTP**) indicates a lack of such a secure connection, and therefore a potential vulnerability [3].

Second, the use of many subdomains is correlated with illegitimacy. In the Kaggle Malicious URLs dataset, 3.76% of benign URLs contain at least three subdomains, while a much greater 9.46% of malicious URLs do. This reflects the fact that a malicious URL might try to mimic a benign one by including similar parts; for example, a malicious webhost could claim the subdomain “real.official.google.xyz.com” (domain=”xyz”, subdomains=”official.google”), hoping that someone mistakes it for the benign “google.com” domain.

Third, the TLD is highly indicative of a URL’s legitimacy. For example, in the dataset we have chosen, there are nearly 400,000 **.com** websites out of 650,000 total URLs. Of these **.com** websites, more than 77% are benign, despite the dataset being only about 65% benign. On a similar thread, more than 72% of all benign URLs in the dataset end in **.com**, while only 40% of malicious URLs do. Furthermore, 32% of malicious URLs use a rare TLD – one that is shared by less than 1% of the dataset – while only 8% of benign URLs use a rare TLD. The TLD alone unpacks a massive portion of the embedding space, and provides crucial insights into the status of a URL.

In machine learning tasks, feature extraction refers to the process of transforming a dataset into a format that is machine-readable, usually using numbers. In upholding this principle, we intentionally choose to focus on features that can be intuitively represented by simple numbers, like the length of a URL or the

number of subdomains, rather than more qualitative features such as domain name familiarity or readability [4].

In this project, our primary objective is to provide a precise, accurate, and high-recall model that filters out malicious URLs and favors benign ones. By screening out potentially harmful or exploitative websites, we will ultimately make the internet a safer place.

2. Methodology

A. Data Preprocessing

We will utilize state-of-the-art data preprocessing techniques to transform messy URL strings into numeric embeddings that can be understood by a Machine Learning model.

We begin by canonicalizing the dataset, meaning that we transform each URL into a more standardized form to avoid double-counting URLs that point to the same source, as well as reducing overall noise. We accomplish this by stripping ports (anything following a colon that appears after an `http(s)://` prefix, if any), as well as fragments (parts following a pound symbol `#`). We also convert each URL to lower case to avoid missing any associations on account of mismatched capitalization, and especially because the core parts of a URL are case insensitive.

After canonicalizing the dataset, we remove incomplete and duplicate data points from the dataset. In the Kaggle Malicious URLs Dataset, there are exactly 10,066 duplicate entries (about 1.5%), and zero empty/missing entries. We can justify data removal (as opposed to augmentation) in both supervised training/evaluation and unsupervised learning. In a supervised learning pipeline, there are enough valid data points (98.5%) to properly learn patterns, so the missing data do not make a difference. Similarly, when evaluating a data point, the only parameter needed is the URL, which will always be provided in real-world scenarios (since URLs are required to navigate to a webpage). In unsupervised learning approaches, the missing data points will be too few to adversely impact clustering and other forms of association.

Once the missing or duplicate data points are removed, we want to balance the dataset to contain an equal number of malicious and benign URLs. As published, the dataset contains just over 65% benign URLs, and just less than 35% malicious URLs. This imbalance encourages overfitting, which is something we aim to avoid. Therefore, in order to make the dataset more balanced, we randomly delete a subset of the benign URLs until the two labels are equally common.



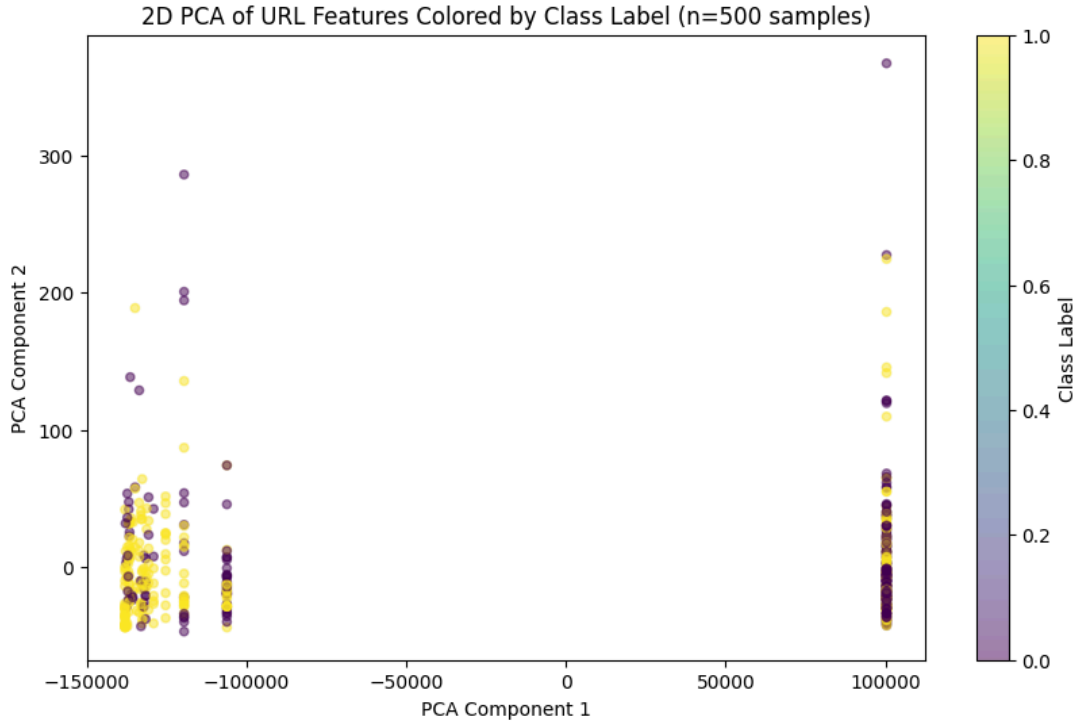
Next, we use string manipulation techniques to isolate the domain (e.g. google, amazon, discord), subdomain (if any) (e.g. docs.google.com, acm.sdsu.edu), and top level domain (TLD) (e.g. .com, .org, .net). We analyze the length and character composition of these components, as well as the number of subdomains and the usage frequency of the TLD, in order to delineate benign URLs from malicious ones. Finally, we parse the URL to determine if it uses a secure (https) or insecure (http) connection, which is encoded accordingly in binary.

In summary, each URL is transformed into the feature space described below:

- $y^{(i)}$: **label** (benign=0, malicious=1)
- $x_1^{(i)}$: **is_https** (boolean)
- $x_2^{(i)}$: **url_length** (integer)
- $x_3^{(i)}$: **subdomain_count** (integer)
- $x_4^{(i)}$: **count_special_chars** (integer)
- $x_5^{(i)}$: **digit_to_length_ratio** (real number in range $[0., 1.]$)
- $x_6^{(i)}$: **TLD_usage_frequency** (integer)

Basic correlation analysis on the dataset reveals mixed levels of correlation between each feature and the data point's corresponding class. No individual feature demonstrates strong correlation with the output, but when combined, these features produce meaningful associations. In order to preserve the most crucial patterns while reducing the complexity of our feature embedding, we perform Principal Component Analysis (PCA), transforming our 6-dimensional representation into just two highly-relevant parameters.

A subset of the post-PCA dataset is visualized in two dimensions in the chart below, where a purple dot represents a benign URL and a yellow dot represents a malicious URL. We downsample to 500 instances to make the data more visually decipherable, while preserving the general distribution of data points.



Basic correlation coefficient analysis reveals that PC1 mainly encodes domain-related factors such as TLD usage frequency ($r=1.000$) and subdomain count ($r=-0.195$), while PC2 mainly encodes lexical features including URL length ($r=0.999$), special character count ($r=0.781$), and digit-to-length ratio ($r=0.139$). The vast difference in scaling between PC1 ($\sim 100,000$) and PC2 (~ 300) indicates that a URL's (full) domain is its single most important component, whereas lexical features matter to a lesser extent.

B. Model Selection

In this project, we use three different machine learning approaches to predict the label (benign or malicious) of URLs. In particular, we choose Logistic Regression, Recurrent Neural Networks (RNNs), and K-Nearest Neighbors (KNN).

> I. Logistic Regression

The rationale behind Logistic Regression is that it takes a series of inputs, such as URL embeddings, and it produces a binary output. In other words, Logistic Regression implements binary classification, which is exactly the kind of task we are trying to solve here. We convert each URL into a feature-space embedding, train the LogReg model to find the mapping between these embeddings and some unbounded numerical output, and then use a Sigmoid function to regularize the output to a binary value according to a specified threshold.

Logistic Regression is characterized by its low complexity, in both time and space, making it highly efficient to train (roughly linear) and run inference (roughly constant) with large datasets (such as the one we chose in this project) that contain many features (unlike our 2-component embedding). LogReg is also highly interpretable, because it outputs a binary 0 or 1, which directly corresponds to a class label.

Furthermore, LogReg is highly nonlinear, allowing the logistic decision boundary to mould itself around the data distribution, making it applicable to a much wider range of datasets and classification problems. To implement LogReg, we use scikit-learn's built-in LogisticRegression class and use the default solver `lbfgs` with regularization strength `C=1.0` and an L2 loss.

> II. Recurrent Neural Network

Additionally, we use Recurrent Neural Networks (RNNs) because this type of model can take sequences of data, such as text, sentences, and URLs, for sentiment analysis and recognizing named entities. In this case, we can use an RNN to take a list of URLs as inputs and predict each URL's label by allowing it to recognize common subdomains, names, and TLDs that appear in many benign and malicious URLs. With its unique nature, this type of model only needs to do the first step of the data preprocessing technique, where we lowercase all of the links, so that it can accurately predict the legitimacy of a wide variety of links!

For clarification, we must further explain how the unique complex nature of RNN models can determine the validity of URLs from text alone. RNNs are separated into three main layers: the input layer, the hidden layer, and the output layer. First, in the input layer, we initially start with a 3D vector of dimensions `[n_chars, 1, n_categories]`, where `n_chars` represents the number of unique characters throughout the entire dataset, and `n_categories` represents the number of types of URLs (benign, defacement, phishing, and malware). This vector will count how much of each character occurs for each URL that the model is trained on. In other words, via **frequency encoding**. Next, in the hidden layer, we specify some arbitrary amount of hidden layers with `n_hidden`, in this case 128. The hidden layer transforms the initial 3D vector input using matrix multiplication with the model's parameters, then utilizes an activation function on the resulting matrix to give to the output layer. In this case, we use the hyperbolic tangent function as our activation function for our output layer. Then, the model reuses this result from the output layer as input for the hidden layer. The process of the hidden layer is reiterated until it has been performed `n_hidden` times. Finally, with the last output that is received, the elements of the 3D array are fully connected then put through a softmax layer to receive the final `1 x n_categories` vector output, with the index of the max value indicating the model's prediction on what type the URL is.

For further optimization, there are a few extra hyperparameters necessary to train our RNN. In addition to our originally planned evaluation metrics, we also utilized a negative-log-likelihood loss function and stochastic gradient descent to measure and fine tune the model's performance, along with learning rate `alpha`. This was implemented with `nn.NLLLoss()` and `torch.optim.SGD()` respectively.

Unfortunately, there is one major drawback of this model that cannot be overlooked. With a training model this complex, the training function requires us to iterate through N samples, and each output received from the model takes K iterations of hidden layers to get. This leads to $O(NK)$ runtime, which is extremely slow especially when we have to consider two variables. Furthermore, the model needs 3D frequency vectors for each of the N samples to understand the URLs it is dealing with. Assuming that the vector sizes are *constant*, the model practically deals with an $O(N)$ space complexity, which can be moderately concerning for a dataset size like ours.

> III. K-Nearest Neighbors

Finally, we use K-Nearest Neighbors (KNN) because it provides a scaffold for unsupervised discrete classification, and in particular, binary classification. In essence, any given URL can be compared to others in the embedding space. If the URL's nearest neighbors are mostly malicious, then we will assume that our URL is also malicious. Likewise, if the URL's nearest neighbors are mostly benign, then we can assume that our target URL is also benign.

When constructing the KNN model, it was important to ensure that hyperparameters were set in order to prevent overfitting. Because of this, we need to ensure that an optimal k (or number of nearby points used to make a prediction) is chosen before training the model. This can be done by cross-validating a range of points in order to check for the number of neighbors that would result in the highest accuracy. In `knn.ipynb`, k values ranging from 1 to 30 are tested, with average best k values being about 6 and 8.

When constructing the model, different distance metrics in order to identify the nearest neighbor were also tested to see if that would change the accuracy. Both the Euclidean Distance and Manhattan Distance formulas were tested, and while the difference isn't significant, using the Manhattan formula provides slightly higher accuracy by about 0.02%. This might be due to the fact that in the sample KNN distribution graph, points tend to cluster in straight vertical lines, since the Manhattan formula is typically used for calculating distance in grid-like patterns. This is specified in the calling of `knn_classifier = KNeighborsClassifier(n_neighbors=k, p=1, weights='distance')`. The function called by default uses the Minkowski Distance formula, which acts as the Manhattan or Euclidean formula depending on the parameter p is set to either 1 or 2 respectively. p is then set to 1 to ensure the use of the Manhattan formula.

The weight function has also been changed from the default 'uniform' to distance. This means that for each datapoint being equally weighted, the points will have more influence on a prediction based on how close they are to each other. A majority of the points in the distribution are linear, vertical clusters with very few outliers, so setting weight to 'distance' yields a much greater accuracy. For the algorithm used, the default 'auto' is used since that decides an optimal algorithm based on the values passed in, which allows for better and more consistent results.

KNN is what's known as a lazy algorithm, meaning that the training data is only stored until a prediction is made, so the process of training the algorithm runs at near constant time. However, this does result in the prediction time taking a lot longer on average. Not only that, but storing the entirety of the dataset takes a lot of space. For prediction, the runtime is dependent on the size of the dataset and the number of features.

The final output of the KNN prediction graph is fairly easy to interpret. Because prediction relies on the classification of nearby points, it is usually easy to distinguish one cluster from another. This also means there is no obvious distribution for the data since the classification of data is dependent on its closest neighbors instead of any direct variables, so KNN is also considered to be a nonlinear algorithm.

C. Evaluation Metrics

For our evaluation metrics, we will calculate **accuracy, precision, recall, and F1 score** for each model we train. We will accomplish this by determining the **confusion matrix** (True Positives, False Positive, True Negatives, and False Negatives) produced by the models, and then using widely-accepted formulas to calculate the aforementioned evaluation metrics. This will allow us to see how well our models work in catching malicious URLs, while allowing benign URLs to pass through.

3. Results

A. Logistic Regression (LogReg)

With the standard decision probability threshold of 0.5, we get the following results:

(P=0.50) Confusion Matrix (Negative=Benign, Positive=Malicious)

	Negative Prediction	Positive Prediction
Negative Ground Truth	TN: 30562 (35.9%)	FP: 11961 (14.0%)
Positive Ground Truth	FN: 17061 (20.0%)	TP: 25634 (30.1%)

(P=0.50) Evaluation Metrics

Accuracy	Precision	Recall	FPR	F1 Score
0.66	0.68	0.60	0.28	0.64

Reducing the threshold to 0.35 increases the chance of a URL being classified as positive (malicious), which improves recall, and thereby improves the chances that the model will catch a malicious website, at the cost of more benign websites being falsely flagged as malicious. In that case, the results are as follows.

(P=0.35) Confusion Matrix (Negative=Benign, Positive=Malicious)

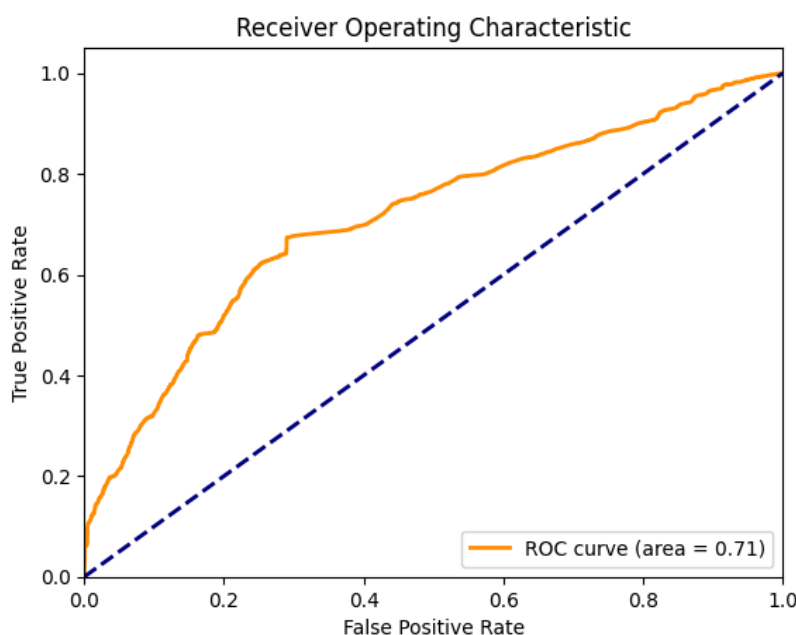
	Negative Prediction	Positive Prediction
Negative Ground Truth	TN: 13139 (15.4%)	FP: 29384 (34.5%)
Positive Ground Truth	FN: 6283 (7.4%)	TP: 36412 (42.7%)

(P=0.35) Evaluation Metrics

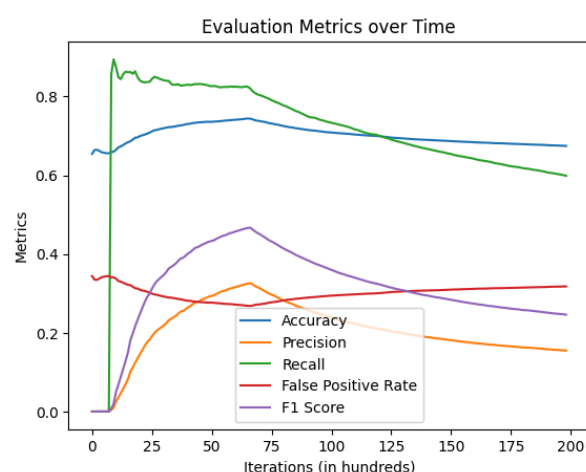
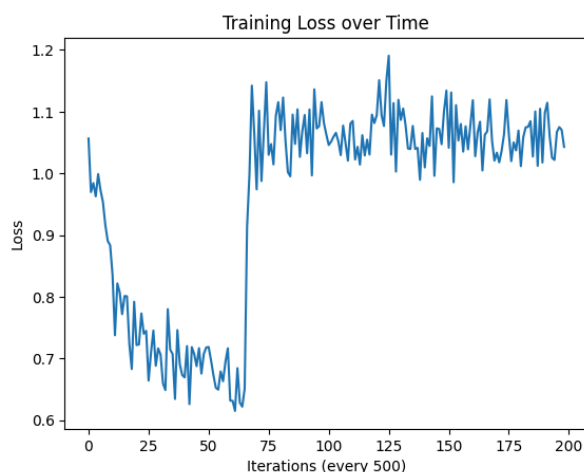
Accuracy	Precision	Recall	FPR	F1 Score
0.58	0.55	0.85	0.69	0.67

Notice how the FPR dramatically increased from 0.28 to 0.69, indicating that the majority of benign websites will be falsely flagged as malicious. This is not ideal, and would make web browsing very difficult, although malicious websites would be more likely to be flagged.

The **Receiver Operating Characteristic** (ROC) curve below illustrates the full tradeoff between the False Positive Rate (FPR) and the True Positive Rate (Recall) for various classification thresholds. Notice how there is a rough point of maximal concavity where $FPR \sim 0.3$ and $TPR \sim 0.6$, which aligns very closely with their respective values when the classification threshold is the standard 0.50 ($FPR = 0.28$ and $TPR = 0.60$). This suggests that our model does a reasonably good job balancing performance for malicious and benign URLs.



B. Recurrent Neural Network (RNN)



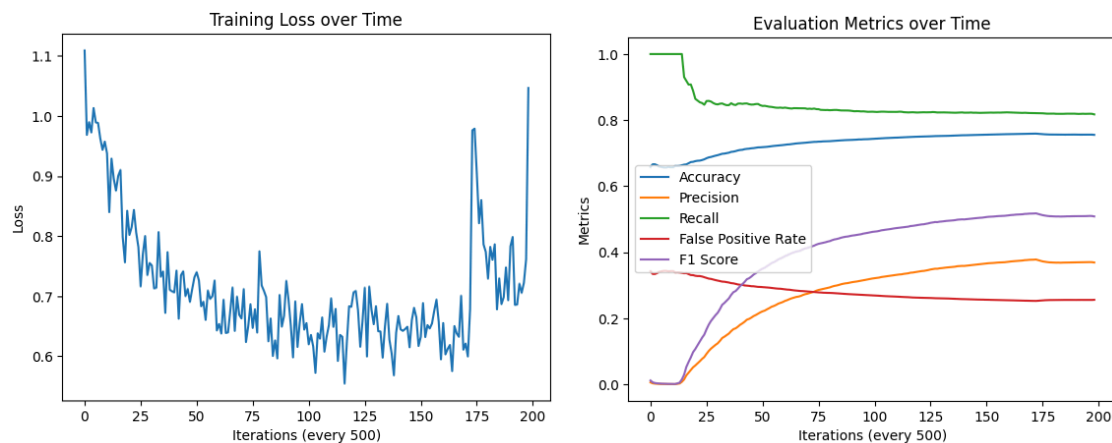
For the RNN model, we have two runs with different learning rates with 128 hidden layers throughout both runs. First, 100,000 samples and a learning rate of 0.005, the loss function trends downwards, then remains stagnant at higher values. Furthermore, the accuracy, precision, recall, and F1 score decline and the false positive rate increases, meaning that the model regresses over time. These results are likely due to uncommon cases where shorter URLs are malicious and longer URLs are benign. Another potential reason for the loss function acting like this may be due to the learning rate being too large, hence the stagnation. So, during the second run, I decided to halve the initial learning rate.

Confusion Matrix at $\alpha = 0.005$ (Negative=Benign, Positive=Malicious)

	Negative Prediction	Positive Prediction
Negative Ground Truth	TN: 62204 (62.20%)	FP: 28963 (28.96%)
Positive Ground Truth	FN: 3547 (3.55%)	TP: 5286 (5.29%)

Evaluation Metrics at $\alpha = 0.005$

Accuracy	Precision	Recall	FPR	F1 Score
0.67	0.15	0.60	0.32	0.25



As expected, lowering the learning rate from 0.005 to 0.0025 led to a mostly smoother decline! Additionally, while the precision only increased slightly, both the recall and accuracy of the model increased significantly, indicating that the model has improved its predictions and is able to save a good majority of users from malicious links! Once again, disregard the loss spikes at the end, as they are likely rare cases of URLs being either short and malicious or lengthy and benign.

Confusion Matrix at $\alpha = 0.0025$ (Negative=Benign, Positive=Malicious)

	Negative Prediction	Positive Prediction
Negative Ground Truth	TN: 62932 (62.93%)	FP: 21663 (21.66%)

Positive Ground Truth	FN: 2819 (2.82%)	TP: 12586 (12.59%)
-----------------------	------------------	---------------------------

Evaluation Metrics at $\alpha = 0.025$

Accuracy	Precision	Recall	FPR	F1 Score
0.76	0.37	0.82	0.26	0.51

C. K-Nearest Neighbors (KNN)

Confusion Matrix (Negative=Benign, Positive=Malicious)

	Negative Prediction	Positive Prediction
Negative Ground Truth	TN: 36585 (42.93%)	FP: 5973 (7.01%)
Positive Ground Truth	FN: 10668 (12.91%)	TP: 32008 (37.15%)

Evaluation Metrics (Benign, 0)

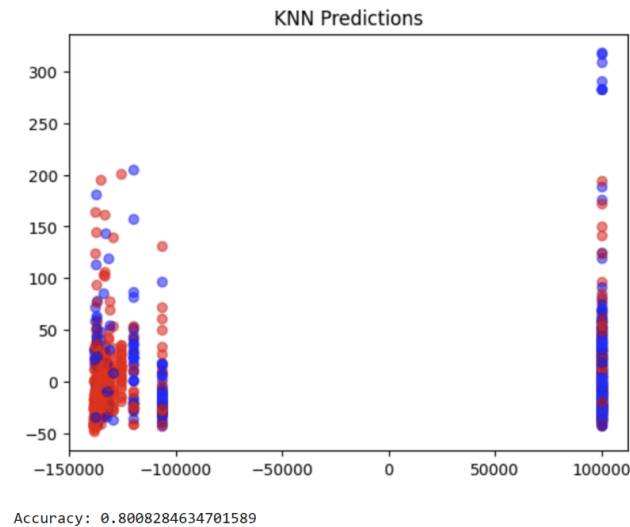
Accuracy	Precision	Recall	FPR	F1 Score
0.80	0.77	0.86	0.14	0.81

Evaluation Metrics (Malicious, 1)

Accuracy	Precision	Recall	FPR	F1 Score
0.80	0.84	0.74	0.14	0.79

The evaluation metrics differ depending on the class, in this case being benign (0) and malignant (1).

NOTE: red indicates a malignant prediction, blue indicates a benign prediction



4. Discussion and Conclusion

In this project, we attempted to predict whether a URL points to a malicious or benign website by employing a variety of supervised machine learning techniques. We began by transforming a large dataset of URLs into a machine-readable feature embedding. We used Logistic Regression, a staple of binary classification tasks, to classify URLs based on a sigmoid curve fit to the featurized data, resulting in better-than-random but less-than-ideal performance. Then, we employed a Recurrent Neural Network (RNN) that parses through each URL one character at a time, yielding similar performance to Logistic Regression, but using a different strategy. Finally, we applied K-Nearest Neighbors (KNN), a technique that classifies each URL based on the labeled URLs that are closest to it in the feature space, which produced significantly stronger results.

The overperformance of KNN compared to LogReg and RNN speaks to the nonlinearity and non-separability of the data. KNN benefits from the fact that it uses the identities of nearby points when predicting the identity of a given point, rather than trying to develop some mathematical function that estimates a URL's inherent probability of being malicious. Indeed, there is no single feature or dimension that is solely correlated with the benignness of a URL. Specific combinations of features, rather, are far more influential.

Future work will explore additional supervised classification methods including classification trees and convolutional neural networks, as well as unsupervised approaches such as K-means clustering and DBSCAN clustering. With this in mind, it is important to recognize that a URL cannot always be accurately classified solely based on the lexical symbols that comprise it. Many safe-looking URLs will ultimately be malicious, and many suspicious URLs will be completely safe in reality. In practice, a tool meant to classify websites as malicious or benign should use actual information from the website, as well as external factors such as user reports, rather than simply comparing the URL to others. Crucially, users should understand that these predictions are imperfect, and that when they use the internet, they do so at their own risk.

5. Individual Contributions

Each group member was responsible for developing one model each. In addition, each group member was responsible for ensuring that the project and this report remains organized, with each section they are assigned to write being dictated by the model they have chosen to develop. Other portions of this report will be assigned to group members accordingly.

Below are the individual contributions of each group member:

AJ Gabriel:

- Developed an RNN model and preprocessed malicious URL dataset to easily organize and manipulate data to eliminate uppercase characters
- Saved results of RNN model in two pickle files
- Created two evaluation models measuring the performance of two runs of the RNN model
- Compressed page length of the project report with Times New Roman, and polished format of the report

Talia Goody:

- Developed, executed, and analyzed a KNN model (`knn.ipynb`)
- Drafted section 5 (Individual Contributions)

Everett Richards:

- Developed dataset pre-processing pipeline (`process_dataset.py`)
- Developed, executed, and analyzed a logistic regression model (`LogReg.ipynb`)
- Optimized code of RNN model to utilize frequency encoding rather than one-hot encoding
- Drafted abstract, section 1 (Introduction), section 2 (Methodology, except parts specifically pertaining to RNN and KNN), 3A (LogReg results), Conclusion
 - Revised sections based on grader feedback from the project proposal
- Drafted `README.md`

References

[1] Clifford Chi. *Parts of a URL: A Short Guide*. HubSpot, 2021.
<https://blog.hubspot.com/marketing/parts-url>

[2] Manu Siddhartha. *Malicious URLs dataset*. Kaggle, 2021.
<https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset>

[3] Amazon Web Services. *What's the Difference Between HTTP and HTTPS?*
<https://aws.amazon.com/compare/the-difference-between-https-and-http/>

[4] Apoorva Joshi, Levi Lloyd, Paul Westin, Srini Seethapathy. *Using Lexical Features for Malicious URL Detection – A Machine Learning Approach*. arXiv, 2019.